Parametric Insurance MGA Implementation Guide

Parametric Insurance MGA Implementation Guide with TrustWeave

> **Building "Atlas Parametric" - An EO-Driven Parametric Insurance MGA**

This comprehensive guide shows you how to build your parametric insurance MGA solution using TrustWeave as the trust and integrity foundation. This implementation covers SAR flood, heatwave, solar attenuation, hurricane, and drought parametric products with instant payouts and objective EO triggers.

Executive Summary

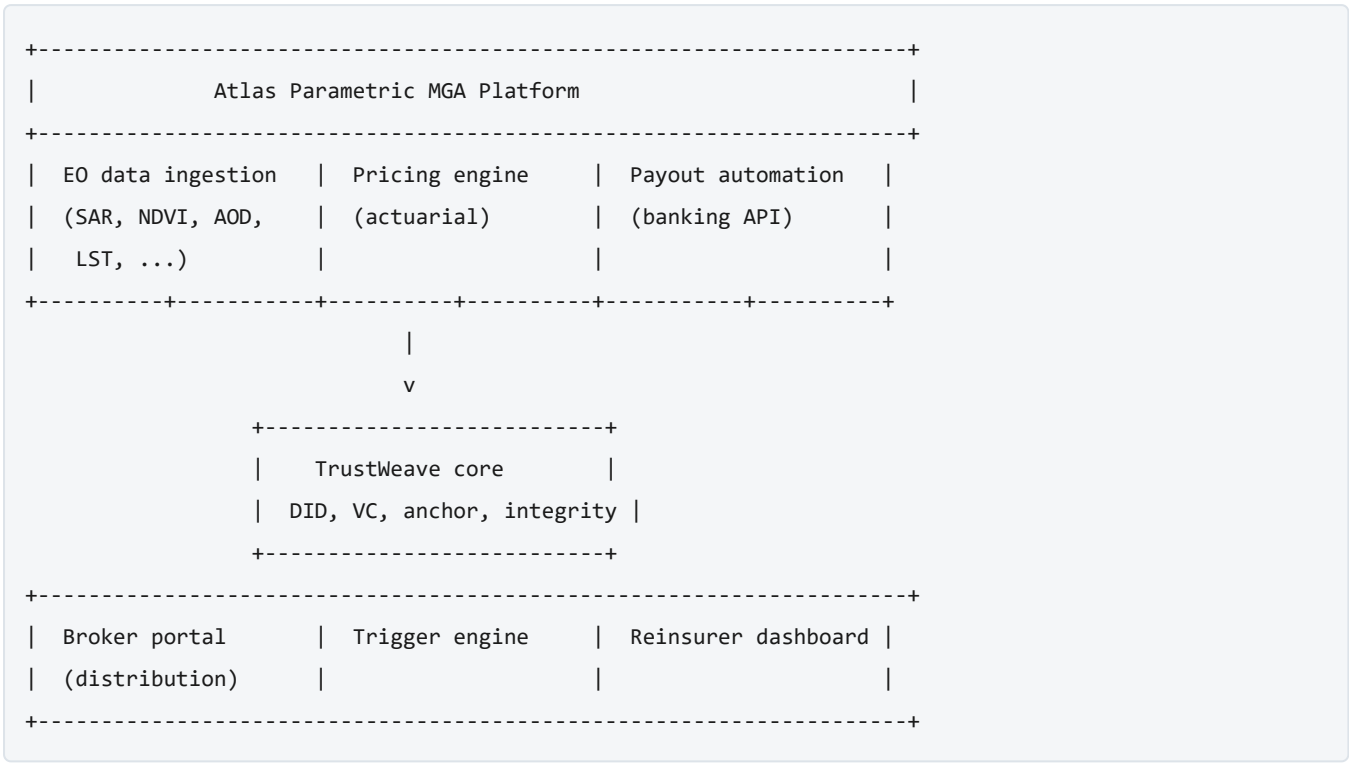
What You're Building:

- A parametric insurance MGA (Managing General Agent) platform
- EO-driven triggers (SAR, NDVI, AOD, LST, InSAR)
- Automated 24-72 hour payouts
- Multi-provider EO data acceptance
- Tamper-proof trigger verification
- Regulatory-compliant audit trails

Why TrustWeave:

- **Trust Foundation**: Verifiable Credentials for EO data integrity
- **Multi-Provider Support**: Accept data from ESA, Planet, NASA, NOAA without custom integrations
- **Blockchain Anchoring**: Tamper-proof trigger records for regulatory compliance
- **Standardization**: W3C-compliant format works across all providers
- **Automation**: Enable instant payouts with verifiable triggers

Architecture Overview



Core Components

1. TrustWeave core

TrustWeave provides:

- **DID Management**: Identity for insurers, EO providers, reinsurers, brokers
- **Verifiable Credentials**: EO data wrapped in VCs for integrity
- **Blockchain Anchoring**: Tamper-proof trigger records
- **Multi-Provider Support**: Accept EO data from any certified provider

2. EO Data Ingestion Pipeline

Processes:

- Sentinel-1 SAR (flood detection)
- MODIS/VIIRS LST (heatwave)
- AOD + irradiance (solar attenuation)
- NDVI (drought/agriculture)
- GPM/IMERG (rainfall)
- InSAR (deformation)

3. Trigger Engine

Evaluates parametric triggers:

- Real-time EO data ingestion
- Threshold evaluation
- Tiered payout calculation
- Automatic trigger verification

4. Pricing Engine

Actuarial pricing:

- EO climate archive (20-40 years)
- Hazard-frequency modeling
- Geographic risk scoring
- Reinsurer-approved rates

5. Payout Automation

Automated payouts:

- KYC/AML integration
- Banking API integration
- Reinsurer notification
- Audit trail generation

Implementation: Product Suite

Product 1: SAR-Based Flood Parametric

Data Sources: Sentinel-1 SAR + DEM **Markets:** US (NC, SC, FL, GA) **Payout Range:** $25k - $5M **Triggers:** Depth thresholds (20cm, 50cm, 1m)

Implementation

```
package com.atlasparametric.products.flood

import org.trustweave.trust.TrustWeave
import org.trustweave.contract.models.*
import org.trustweave.core.*
import org.trustweave.core.util.DigestUtils
import org.trustweave.core.json.jsonData
import java.time.Instant
import org.trustweave.core.Did
import org.trustweave.trust.types.getOrThrow
import org.trustweave.did.resolver.DidResolutionResult
import org.trustweave.did.resolver.errorMessage
import org.trustweave.did.identifiers.extractKeyId
import org.trustweave.credential.results.getOrThrow

/**
 * SAR Flood Parametric Product using Smart Contracts
 *
 * Uses Sentinel-1 SAR data to detect flood depth and trigger automatic payouts
 */
class SarFloodProduct(
    private val TrustWeave: TrustWeave,
    private val eoProviderDid: String
) {

    /**
     * Create a flood insurance contract
     */
    suspend fun createFloodContract(
        insurerDid: String,
        insuredDid: String,
        coverageAmount: Double,
        location: Location
    ): SmartContract {

        val contract = trustWeave.contracts.draft {
            contractType = ContractType.Insurance
            executionModel = ExecutionModel.Parametric(
                triggerType = TriggerType.EarthObservation,
                evaluationEngine = "parametric-insurance"
            )
            parties = ContractParties(
                primaryPartyDid = insurerDid,
                counterpartyDid = insuredDid
```

```
        )
        terms = ContractTerms(
            obligations = listOf(
                Obligation(
                    id = "payout-obligation",
                    partyDid = insurerDid,
                    description = "Pay out based on flood depth tier",
                    obligationType = ObligationType.PAYMENT
                )
            ),
            conditions = listOf(
                ContractCondition(
                    id = "flood-threshold-20cm",
                    description = "Flood depth >= 20cm (Tier 1)",
                    conditionType = ConditionType.THRESHOLD,
                    expression = "$.floodDepthCm >= 20"
                ),
                ContractCondition(
                    id = "flood-threshold-50cm",
                    description = "Flood depth >= 50cm (Tier 2)",
                    conditionType = ConditionType.THRESHOLD,
                    expression = "$.floodDepthCm >= 50"
                ),
                ContractCondition(
                    id = "flood-threshold-100cm",
                    description = "Flood depth >= 100cm (Tier 3)",
                    conditionType = ConditionType.THRESHOLD,
                    expression = "$.floodDepthCm >= 100"
                )
            ),
            jurisdiction = "US",
            governingLaw = "State of North Carolina"
        )
        effectiveDate = Instant.now().toString()
        expirationDate = Instant.now().plusSeconds(365 * 24 * 60 * 60).toString()
        contractData {
            "productType" to "SarFlood"
            "coverageAmount" to coverageAmount
            "location" {
                "latitude" to location.latitude
                "longitude" to location.longitude
                "address" to location.address
                "region" to location.region
            }
            "thresholds" {
                "tier1Cm" to 20.0
                "tier2Cm" to 50.0
```

```
                "tier3Cm" to 100.0
            }
            "payoutTiers" {
                "tier1" to 0.25  // 25% of coverage
                "tier2" to 0.50  // 50% of coverage
                "tier3" to 1.0   // 100% of coverage
            }
        }
    }.getOrThrow()

    println("[OK] Flood contract draft created: ${contract.id}")
    return contract
}

/**
 * Bind contract (issue VC and anchor to blockchain)
 */
suspend fun bindFloodContract(
    contract: SmartContract,
    insurerDid: String,
    insurerKeyId: String
): BoundContract {

    val bound = trustWeave.contracts.bindContract(
        contractId = contract.id,
        issuerDid = insurerDid,
        issuerKeyId = insurerKeyId,
        chainId = "algorand:mainnet"
    ).getOrThrow()

    println("[OK] Contract bound: ${bound.credentialId}, anchored: ${bound.anchorRef.txHash}")
    return bound
}

/**
 * Process SAR flood data and issue verifiable credential
 */
suspend fun processSarFloodData(
    location: Location,
    sarData: SarFloodMeasurement,
    timestamp: Instant
): Result<VerifiableCredential> = trustweaveCatching {

    // Step 1: Create EO data payload using the typed DSL
    val floodData = jsonData {
        "id" to "sar-flood-${location.id}-${timestamp.toEpochMilli()}"
        "type" to "SarFloodMeasurement"
```

```
            "location" {
                "latitude" to location.latitude
                "longitude" to location.longitude
                "address" to location.address
                "region" to location.region
            }
            "measurement" {
                "floodDepthCm" to sarData.floodDepthCm
                "floodAreaSqKm" to sarData.floodAreaSqKm
                "confidence" to sarData.confidence
                "source" to "Sentinel-1 SAR"
                "processingMethod" to "SAR Flood Extraction"
                "demUsed" to sarData.demUsed
                "timestamp" to timestamp.toString()
            }
            "quality" {
                "validationStatus" to "validated"
                "dataQuality" to sarData.quality
            }
        }

        // Step 2: Compute data digest for integrity
        val dataDigest = DigestUtils.sha256DigestMultibase(floodData)

        // Step 3: Issue verifiable credential for EO data
        // Note: eoProviderDid is a String (DID value), so we need to create a Did object for
         resolveDid

        val eoProviderDidObj = Did(eoProviderDid)
        val eoProviderDoc = when (val res = trustWeave.resolveDid(eoProviderDidObj)) {
            is DidResolutionResult.Success -> res.document
            else -> throw IllegalStateException(res.errorMessage ?: "Failed to resolve DID")
        }
        val eoProviderKeyId = eoProviderDoc.verificationMethod.firstOrNull()?.extractKeyId()
            ?: throw IllegalStateException("No verification method found")

        val floodIssuanceResult = trustWeave.issue {
            credential {
                id("urn:eo:sar-flood:${location.id}-${timestamp.toEpochMilli()}")
                type("VerifiableCredential", "EarthObservationCredential", "InsuranceOracleCredential",
         "SarFloodCredential")
                issuer(eoProviderDid)
                subject {
                    id("urn:eo:sar-flood:${location.id}-${timestamp.toEpochMilli()}")
                    "dataType" to "SarFloodMeasurement"
                    "data" to floodData
                    "dataDigest" to dataDigest
                    "provider" to eoProviderDid
```

```
                "timestamp" to timestamp.toString()
            }
            issued(Instant.now())
        }
        by(issuerDid = eoProviderDid, keyId = eoProviderKeyId)
    }

    val floodCredential = floodIssuanceResult.getOrThrow()

    // Step 4: Anchor to blockchain for tamper-proof record
    val anchored = trustWeave.blockchains.anchor(
        data = floodCredential,
        serializer = VerifiableCredential.serializer(),
        chainId = "algorand:mainnet"
    )
    println("[OK] SAR Flood Credential issued and anchored: ${anchored.ref.txHash}")

    floodCredential
}

/**
 * Execute contract with flood data
 */
suspend fun executeFloodContract(
    contract: SmartContract,
    floodCredential: VerifiableCredential
): ExecutionResult {

    // Extract flood depth from credential
    val credentialSubject = floodCredential.credentialSubject
    val floodDepthCm = credentialSubject.jsonObject["data"]
        ?.jsonObject?.get("measurement")
        ?.jsonObject?.get("floodDepthCm")
        ?.jsonPrimitive?.content?.toDouble()
        ?: error("Flood depth not found in credential")

    // Execute contract with trigger data
    val result = trustWeave.contracts.executeContract(contract) {
        trigger {
            "floodDepthCm" to floodDepthCm
            "credentialId" to floodCredential.id
            "timestamp" to Instant.now().toString()
        }
    }.getOrThrow()

    if (result.executed) {
        println("[OK] Contract executed! Payout triggered for flood depth: ${floodDepthCm}cm")
```

```
            result.outcomes.forEach { outcome ->
                println("   Outcome: ${outcome.description}")
                outcome.monetaryImpact?.let {
                    println("   Amount: ${it.amount} ${it.currency}")
                }
            }
        } else {
            println("[WARN]  Contract conditions not met (flood depth: ${floodDepthCm}cm)")
        }

        return result
    }
}

// Data Models
data class Location(
    val id: String,
    val latitude: Double,
    val longitude: Double,
    val address: String,
    val region: String
)

data class SarFloodMeasurement(
    val floodDepthCm: Double,
    val floodAreaSqKm: Double,
    val confidence: Double,
    val demUsed: String,
    val quality: String
)
```

Product 2: Heatwave Parametric

Data Sources: MODIS LST + ERA5 **Markets:** GCC (Saudi Arabia, UAE) **Triggers:** > XÃ, C for Y days **Clients:** Construction, energy, government

Implementation

```
package com.atlasparametric.products.heatwave

import org.trustweave.trust.TrustWeave
import org.trustweave.core.*
import org.trustweave.core.util.DigestUtils
import org.trustweave.core.json.jsonData
import java.time.Instant
import java.time.Duration
import org.trustweave.core.Did

class HeatwaveProduct(
    private val TrustWeave: TrustWeave,
    private val eoProviderDid: String
) {

    /**
     * Process heatwave data from MODIS LST
     */
    suspend fun processHeatwaveData(
        location: Location,
        lstData: List<LstMeasurement>,
        threshold: HeatwaveThreshold
    ): Result<VerifiableCredential> = trustweaveCatching {

        // Calculate consecutive days above threshold
        val consecutiveDays = calculateConsecutiveDays(lstData, threshold.temperatureC)

        val heatwaveData = jsonData {
            "id" to "heatwave-${location.id}-${Instant.now().toEpochMilli()}"
            "type" to "HeatwaveMeasurement"
            "location" {
                "latitude" to location.latitude
                "longitude" to location.longitude
                "region" to location.region
            }
            "measurement" {
                "maxTemperatureC" to lstData.maxOf { it.temperatureC }
                "avgTemperatureC" to lstData.average { it.temperatureC }
                "consecutiveDays" to consecutiveDays
                "thresholdC" to threshold.temperatureC
                "minDaysRequired" to threshold.minDays
                "source" to "MODIS LST + ERA5"
                "timestamp" to Instant.now().toString()
            }
```

```
    }

    val dataDigest = DigestUtils.sha256DigestMultibase(heatwaveData)

    // Resolve DID from string (eoProviderDid is a DID string)
    val eoProviderDidObj = Did(eoProviderDid)
    val eoProviderDoc = when (val res = trustWeave.resolveDid(eoProviderDidObj)) {
        is DidResolutionResult.Success -> res.document
        else -> throw IllegalStateException(res.errorMessage ?: "Failed to resolve DID")
    }
    val eoProviderKeyId = eoProviderDoc.verificationMethod.firstOrNull()?.extractKeyId()
        ?: throw IllegalStateException("No verification method found")

    val heatwaveIssuanceResult = trustWeave.issue {
        credential {
            id("urn:eo:heatwave:${location.id}-${Instant.now().toEpochMilli()}")
            type("EarthObservationCredential", "InsuranceOracleCredential", "HeatwaveCredential")
            issuer(eoProviderDid)
            subject {
                id("urn:eo:heatwave:${location.id}-${Instant.now().toEpochMilli()}")
                "dataType" to "HeatwaveMeasurement"
                "data" to heatwaveData
                "dataDigest" to dataDigest
                "provider" to eoProviderDid
                "timestamp" to Instant.now().toString()
            }
            issued(Instant.now())
        }
        by(issuerDid = eoProviderDid, keyId = eoProviderKeyId)
    }

    val heatwaveCredential = heatwaveIssuanceResult.getOrThrow()

    // Anchor to blockchain
    trustWeave.blockchains.anchor(
        data = heatwaveCredential,
        serializer = VerifiableCredential.serializer(),
        chainId = "algorand:mainnet"
    )

    heatwaveCredential
}

/**
 * Create heatwave insurance contract
 */
suspend fun createHeatwaveContract(
```

```
    insurerDid: String,
    insuredDid: String,
    basePayout: Double,
    threshold: HeatwaveThreshold,
    location: Location
): SmartContract {

    val contract = trustWeave.contracts.draft {
        contractType = ContractType.Insurance
        executionModel = ExecutionModel.Parametric(
            triggerType = TriggerType.EarthObservation,
            evaluationEngine = "parametric-insurance"
        )
        parties = ContractParties(
            primaryPartyDid = insurerDid,
            counterpartyDid = insuredDid
        )
        terms = ContractTerms(
            obligations = listOf(
                Obligation(
                    id = "heatwave-payout",
                    partyDid = insurerDid,
                    description = "Pay out for consecutive days above threshold",
                    obligationType = ObligationType.PAYMENT
                )
            ),
            conditions = listOf(
                ContractCondition(
                    id = "heatwave-threshold",
                    description = "Temperature >= ${threshold.temperatureC}°C for
 ${threshold.minDays} days",
                    conditionType = ConditionType.COMPOSITE,
                    expression = "$.consecutiveDays >= ${threshold.minDays} && $.maxTemperatureC >=
 ${threshold.temperatureC}"
                )
            )
        )
        effectiveDate = Instant.now().toString()
        expirationDate = Instant.now().plusSeconds(365 * 24 * 60 * 60).toString()
        contractData {
            "productType" to "Heatwave"
            "basePayout" to basePayout
            "threshold" {
                "temperatureC" to threshold.temperatureC
                "minDays" to threshold.minDays
            }
            "location" {
                "latitude" to location.latitude
```

```
                "longitude" to location.longitude
                "region" to location.region
            }
        }
    }.getOrThrow()

    return contract
}

/**
 * Execute heatwave contract
 */
suspend fun executeHeatwaveContract(
    contract: SmartContract,
    heatwaveCredential: VerifiableCredential
): ExecutionResult {

    val credentialSubject = heatwaveCredential.credentialSubject
    val consecutiveDays = credentialSubject.jsonObject["data"]
        ?.jsonObject?.get("measurement")
        ?.jsonObject?.get("consecutiveDays")
        ?.jsonPrimitive?.content?.toInt()
        ?: error("Consecutive days not found")

    val maxTemp = credentialSubject.jsonObject["data"]
        ?.jsonObject?.get("measurement")
        ?.jsonObject?.get("maxTemperatureC")
        ?.jsonPrimitive?.content?.toDouble()
        ?: error("Max temperature not found")

    return trustWeave.contracts.executeContract(contract) {
        trigger {
            "consecutiveDays" to consecutiveDays
            "maxTemperatureC" to maxTemp
            "credentialId" to heatwaveCredential.id
        }
    }.getOrThrow()
}

private fun calculateConsecutiveDays(
    lstData: List<LstMeasurement>,
    thresholdC: Double
): Int {
    var maxConsecutive = 0
    var currentConsecutive = 0

    for (measurement in lstData.sortedBy { it.timestamp }) {
```

```
            if (measurement.temperatureC > thresholdC) {
                currentConsecutive++
                maxConsecutive = maxOf(maxConsecutive, currentConsecutive)
            } else {
                currentConsecutive = 0
            }
        }

        return maxConsecutive
    }
}

data class LstMeasurement(
    val temperatureC: Double,
    val timestamp: Instant
)

data class HeatwaveThreshold(
    val temperatureC: Double,
    val minDays: Int
)

data class HeatwavePolicy(
    val id: String,
    val location: Location,
    val threshold: HeatwaveThreshold,
    val basePayout: Double,
    val dailyIncrement: Double
)
```

Product 3: Solar Attenuation Parametric

Data Sources: AOD (MODIS, VIIRS) + Irradiance (CERES) **Markets:** GCC (Solar farms) **Triggers:** >30% irradiance drop **Clients:** ACWA Power, NEOM, UAE utilities

Implementation

```
package com.atlasparametric.products.solar

import org.trustweave.trust.TrustWeave
import org.trustweave.core.*
import org.trustweave.core.util.DigestUtils
import org.trustweave.core.json.jsonData
import java.time.Instant
import org.trustweave.core.Did

class SolarAttenuationProduct(
    private val TrustWeave: TrustWeave,
    private val eoProviderDid: String
) {

    /**
     * Process solar attenuation data
     */
    suspend fun processSolarAttenuation(
        location: Location,
        aodData: AodMeasurement,
        irradianceData: IrradianceMeasurement
    ): Result<VerifiableCredential> = trustweaveCatching {

        // Calculate attenuation percentage
        val baselineIrradiance = irradianceData.baselineWattPerSqM
        val currentIrradiance = irradianceData.currentWattPerSqM
        val attenuationPercent = ((baselineIrradiance - currentIrradiance) / baselineIrradiance) *
         100.0

        val solarData = jsonData {
            "id" to "solar-attenuation-${location.id}-${Instant.now().toEpochMilli()}"
            "type" to "SolarAttenuationMeasurement"
            "location" {
                "latitude" to location.latitude
                "longitude" to location.longitude
                "solarFarmId" to location.solarFarmId
            }
            "measurement" {
                "aod" to aodData.aodValue
                "baselineIrradiance" to baselineIrradiance
                "currentIrradiance" to currentIrradiance
                "attenuationPercent" to attenuationPercent
                "source" to "MODIS/VIIRS AOD + CERES Irradiance"
                "timestamp" to Instant.now().toString()
            }
```

```
    }

    val dataDigest = DigestUtils.sha256DigestMultibase(solarData)

    // Resolve DID from string (eoProviderDid is a DID string)
    val eoProviderDidObj = Did(eoProviderDid)
    val eoProviderDoc = when (val res = trustWeave.resolveDid(eoProviderDidObj)) {
        is DidResolutionResult.Success -> res.document
        else -> throw IllegalStateException(res.errorMessage ?: "Failed to resolve DID")
    }
    val eoProviderKeyId = eoProviderDoc.verificationMethod.firstOrNull()?.extractKeyId()
        ?: throw IllegalStateException("No verification method found")

    val solarIssuanceResult = trustWeave.issue {
        credential {
            id("urn:eo:solar-attenuation:${location.id}-${Instant.now().toEpochMilli()}")
            type("EarthObservationCredential", "InsuranceOracleCredential",
     "SolarAttenuationCredential")
            issuer(eoProviderDid)
            subject {
                id("urn:eo:solar-attenuation:${location.id}-${Instant.now().toEpochMilli()}")
                "dataType" to "SolarAttenuationMeasurement"
                "data" to solarData
                "dataDigest" to dataDigest
                "provider" to eoProviderDid
                "timestamp" to Instant.now().toString()
            }
            issued(Instant.now())
        }
        by(issuerDid = eoProviderDid, keyId = eoProviderKeyId)
    }

    val solarCredential = solarIssuanceResult.getOrThrow()

    trustWeave.blockchains.anchor(
        data = solarCredential,
        serializer = VerifiableCredential.serializer(),
        chainId = "algorand:mainnet"
    )

    solarCredential
}

/**
 * Evaluate solar attenuation trigger
 */
suspend fun evaluateSolarTrigger(
```

```
        policy: SolarPolicy,
        solarCredential: VerifiableCredential
    ): TriggerResult {

        val verification = trustWeave.verify {
            credential(solarCredential)
        }
        when (verification) {
            is VerificationResult.Valid -> {
                // Credential is valid, continue
            }
            is VerificationResult.Invalid -> {
                return TriggerResult(
                    triggered = false,
                    reason = "Credential invalid: ${verification.allErrors.joinToString()}"
                )
            }
        }

        val credentialSubject = solarCredential.credentialSubject
        val attenuationPercent = credentialSubject.jsonObject["data"]
            ?.jsonObject?.get("measurement")
            ?.jsonObject?.get("attenuationPercent")
            ?.jsonPrimitive?.content?.toDouble()
            ?: return TriggerResult(triggered = false, reason = "Attenuation not found")

        val thresholdPercent = policy.thresholdPercent

        if (attenuationPercent < thresholdPercent) {
            return TriggerResult(
                triggered = false,
                reason = "Attenuation ($attenuationPercent%) below threshold ($thresholdPercent%)"
            )
        }

        // Calculate payout based on attenuation severity
        val excessAttenuation = attenuationPercent - thresholdPercent
        val payoutAmount = when {
            excessAttenuation >= 20 -> policy.coverageAmount * 1.0  // Full payout
            excessAttenuation >= 10 -> policy.coverageAmount * 0.75
            else -> policy.coverageAmount * 0.50
        }

        return TriggerResult(
            triggered = true,
            attenuationPercent = attenuationPercent,
            payoutAmount = payoutAmount,
```

```
            dataCredentialId = solarCredential.id
        )
    }
}

data class AodMeasurement(
    val aodValue: Double,
    val timestamp: Instant
)

data class IrradianceMeasurement(
    val baselineWattPerSqM: Double,
    val currentWattPerSqM: Double,
    val timestamp: Instant
)

data class SolarPolicy(
    val id: String,
    val location: Location,
    val thresholdPercent: Double = 30.0,
    val coverageAmount: Double
)
```

Complete Workflow Example

Complete Flood Insurance Workflow with Smart Contracts

```
import org.trustweave.trust.types.DidCreationResult
import org.trustweave.did.resolver.DidResolutionResult
import org.trustweave.credential.results.IssuanceResult
import org.trustweave.credential.results.VerificationResult
import org.trustweave.trust.types.getOrThrowDid
import org.trustweave.trust.types.getOrThrow
import org.trustweave.did.identifiers.extractKeyId
import org.trustweave.trust.dsl.credential.KmsProviders.IN_MEMORY
import org.trustweave.trust.dsl.credential.KeyAlgorithms.ED25519
import org.trustweave.trust.dsl.credential.DidMethods.KEY
import org.trustweave.credential.results.getOrThrow

suspend fun completeFloodInsuranceWorkflow() {
    // Step 1: Initialize TrustWeave
    val trustWeave = TrustWeave.build {
        keys { provider(IN_MEMORY); algorithm(ED25519) }
        did { method(KEY) { algorithm(ED25519) } }
        anchor {
            chain("algorand:mainnet") {
                provider("algorand")
                options { /* map from algorandClient / environment */ }
            }
        }
    }

    // Step 2: Create DIDs for parties


    val insurerDid = trustWeave.createDid { method(KEY) }.getOrThrowDid()
    val insuredDid = trustWeave.createDid { method(KEY) }.getOrThrowDid()
    val eoProviderDid = trustWeave.createDid { method(KEY) }.getOrThrowDid()

    val insurerDoc = when (val res = trustWeave.resolveDid(insurerDid)) {
        is DidResolutionResult.Success -> res.document
        else -> throw IllegalStateException(res.errorMessage ?: "Failed to resolve DID")
    }
    val insurerKeyId = insurerDoc.verificationMethod.firstOrNull()?.extractKeyId()
        ?: throw IllegalStateException("No key found")

    // Step 3: Initialize product
    val floodProduct = SarFloodProduct(trustWeave, eoProviderDid.value)
```

```
// Step 4: Create contract
val contract = floodProduct.createFloodContract(
    insurerDid = insurerDid.value,
    insuredDid = insuredDid.value,
    coverageAmount = 1_000_000.0,
    location = Location(
        id = "loc-001",
        latitude = 35.2271,
        longitude = -80.8431,
        address = "Charlotte, NC",
        region = "North Carolina"
    )
)

// Step 5: Bind contract (issue VC and anchor)
val bound = floodProduct.bindFloodContract(
    contract = contract,
    insurerDid = insurerDid.value,
    insurerKeyId = insurerKeyId
)

// Step 6: Activate contract
val activeResult = trustWeave.contracts.activateContract(bound.contract.id)
val active = activeResult.getOrThrow()

// Step 7: Process EO data (in production, this comes from EO provider)
val floodCredentialResult = floodProduct.processSarFloodData(
    location = Location(
        id = "loc-001",
        latitude = 35.2271,
        longitude = -80.8431,
        address = "Charlotte, NC",
        region = "North Carolina"
    ),
    sarData = SarFloodMeasurement(
        floodDepthCm = 75.0,
        floodAreaSqKm = 15.5,
        confidence = 0.95,
        demUsed = "SRTM",
        quality = "high"
    ),
    timestamp = Instant.now()
)

val floodCredential = floodCredentialResult.getOrThrow()

// Step 8: Execute contract
```

```
    val executionResult = floodProduct.executeFloodContract(
        contract = active,
        floodCredential = floodCredential
    )

    // Step 9: Process payout (application-specific)
    if (executionResult.executed) {
        processPayout(executionResult)
    }
}
```

Complete System Integration

Main Application

```
package com.atlasparametric

import org.trustweave.trust.TrustWeave
import org.trustweave.trust.dsl.credential.DidMethods.KEY
import org.trustweave.trust.dsl.credential.KeyAlgorithms.ED25519
import org.trustweave.trust.types.getOrThrowDid
import com.atlasparametric.products.flood.SarFloodProduct
import com.atlasparametric.products.heatwave.HeatwaveProduct
import com.atlasparametric.products.solar.SolarAttenuationProduct
import kotlinx.coroutines.runBlocking

/**
 * Atlas Parametric MGA Platform
 *
 * Main application entry point
 */
class AtlasParametricPlatform {

    private val trustWeave: TrustWeave
    private val sarFloodProduct: SarFloodProduct
    private val heatwaveProduct: HeatwaveProduct
    private val solarProduct: SolarAttenuationProduct

    init {
        trustWeave = TrustWeave.build {
            did { method(KEY) { algorithm(ED25519) } }
            anchor {
                chain("algorand:mainnet") {
                    provider("algorand")
                    options {
                        "apiKey" to System.getenv("ALGORAND_API_KEY")
                    }
                }
            }
        }

        val eoProviderDidResult = runBlocking {
            trustWeave.createDid { method(KEY); algorithm(ED25519) }
        }

        val eoProviderDid = eoProviderDidResult.getOrThrowDid()
```

```
        // Initialize products
        sarFloodProduct = SarFloodProduct(trustWeave, eoProviderDid.value)
        heatwaveProduct = HeatwaveProduct(trustWeave, eoProviderDid.value)
        solarProduct = SolarAttenuationProduct(trustWeave, eoProviderDid.value)
    }

    /**
     * Process EO data and execute contracts
     */
    suspend fun processEoDataAndExecute(
        productType: ProductType,
        eoData: Any,
        contracts: List<SmartContract>
    ): List<ExecutionResult> {

        return when (productType) {
            ProductType.SAR_FLOOD -> {
                val floodData = eoData as SarFloodMeasurement
                processFloodExecutions(floodData, contracts)
            }
            ProductType.HEATWAVE -> {
                val heatData = eoData as List<LstMeasurement>
                processHeatwaveExecutions(heatData, contracts)
            }
            ProductType.SOLAR_ATTENUATION -> {
                val solarData = eoData as Pair<AodMeasurement, IrradianceMeasurement>
                processSolarExecutions(solarData, contracts)
            }
        }
    }

    private suspend fun processFloodExecutions(
        floodData: SarFloodMeasurement,
        contracts: List<SmartContract>
    ): List<ExecutionResult> {

        val results = mutableListOf<ExecutionResult>()

        for (contract in contracts.filter {
            it.contractData.jsonObject["productType"]?.jsonPrimitive?.content == "SarFlood"
        }) {
            // Process SAR data and issue credential
            val location = extractLocation(contract)
            val floodCredentialResult = sarFloodProduct.processSarFloodData(
                location = location,
                sarData = floodData,
                timestamp = Instant.now()
```

```
            )

            val floodCredential = floodCredentialResult.getOrNull() ?: run {
                println("Failed to process flood data")
                continue
            }

            // Execute contract
            val executionResult = sarFloodProduct.executeFloodContract(
                contract = contract,
                floodCredential = floodCredential
            )

            if (executionResult.executed) {
                // Process payout
                processPayout(executionResult)
            }

            results.add(executionResult)
        }

        return results
    }

    private fun extractLocation(contract: SmartContract): Location {
        val locationData = contract.contractData.jsonObject["location"]?.jsonObject
            ?: error("Location not found in contract")

        return Location(
            id = locationData["address"]?.jsonPrimitive?.content ?: "unknown",
            latitude = locationData["latitude"]?.jsonPrimitive?.content?.toDouble() ?: 0.0,
            longitude = locationData["longitude"]?.jsonPrimitive?.content?.toDouble() ?: 0.0,
            address = locationData["address"]?.jsonPrimitive?.content ?: "",
            region = locationData["region"]?.jsonPrimitive?.content ?: ""
        )
    }

    private suspend fun processPayout(executionResult: ExecutionResult) {
        // Integrate with banking API (Stripe, Plaid, etc.)
        // Extract payout amount from executionResult.outcomes
        executionResult.outcomes.forEach { outcome ->
            outcome.monetaryImpact?.let { amount ->
                // Execute payout via banking API
                println("Processing payout: ${amount.amount} ${amount.currency}")
            }
        }
    }
```

```
}

enum class ProductType {
    SAR_FLOOD,
    HEATWAVE,
    SOLAR_ATTENUATION,
    HURRICANE,
    DROUGHT
}

data class PayoutResult(
    val policyId: String,
    val amount: Double,
    val status: String,
    val payoutCredentialId: String,
    val timestamp: Instant
)
```

Multi-Provider EO Data Acceptance

One of TrustWeave's key advantages is accepting EO data from multiple providers:

```kotlin
/**
 * Accept EO data from any certified provider
 */
class MultiProviderEoDataService(
    private val TrustWeave: TrustWeave
) {

    private val certifiedProviders = setOf(
        "did:key:esa-provider",
        "did:key:planet-provider",
        "did:key:nasa-provider",
        "did:key:noaa-provider"
    )

    /**
     * Accept EO data credential from any certified provider
     */
    suspend fun acceptEoDataCredential(
        dataCredential: VerifiableCredential
    ): Result<EoData> = trustweaveCatching {

        // Step 1: Verify credential
        val verificationResult = trustWeave.verify {
            credential(dataCredential)
        }

        when (verificationResult) {
            is VerificationResult.Valid -> {
                // Credential is valid, continue
            }
            is VerificationResult.Invalid -> {
                error("Credential verification failed: ${verificationResult.allErrors.joinToString()}")
            }
        }

        // Step 2: Check if provider is certified
        val issuerDid = dataCredential.issuer.firstOrNull()?.id
            ?: error("No issuer found in credential")

        if (!certifiedProviders.contains(issuerDid)) {
            error("Provider $issuerDid is not certified")
        }

        // Step 3: Extract and return data
        val credentialSubject = dataCredential.credentialSubject
        val dataType = credentialSubject.jsonObject["dataType"]?.jsonPrimitive?.content
```

```
        val data = credentialSubject.jsonObject["data"]?.jsonObject

        EoData(
            type = dataType ?: "Unknown",
            data = data ?: jsonData { },
            provider = issuerDid,
            credentialId = dataCredential.id
        )
    }
}

data class EoData(
    val type: String,
    val data: JsonObject,
    val provider: String,
    val credentialId: String
)
```

Broker Portal Integration

```kotlin
/**
 * Broker Portal API
 */
@RestController
@RequestMapping("/api/broker")
class BrokerPortalController(
    private val platform: AtlasParametricPlatform,
    private val pricingEngine: PricingEngine
) {

    /**
     * Get quote for parametric insurance
     */
    @PostMapping("/quote")
    suspend fun getQuote(
        @RequestBody request: QuoteRequest
    ): QuoteResponse {

        val premium = pricingEngine.calculatePremium(
            productType = request.productType,
            location = request.location,
            coverageAmount = request.coverageAmount
        )

        return QuoteResponse(
            premium = premium,
            coverageAmount = request.coverageAmount,
            productType = request.productType
        )
    }

    /**
     * Bind policy
     */
    @PostMapping("/bind")
    suspend fun bindPolicy(
        @RequestBody request: BindRequest
    ): PolicyResponse {

        // Create policy
        val policy = createPolicy(request)

        // Issue policy credential
        val policyCredential = issuePolicyCredential(policy)
```

```
        return PolicyResponse(
            policyId = policy.id,
            policyCredentialId = policyCredential.id,
            status = "BOUND"
        )
    }
}
```

Regulatory Compliance & Audit Trails

```
/**
 * Audit Trail Service using TrustWeave blockchain anchoring
 */
class AuditTrailService(
    private val trustWeave: TrustWeave
) {

    /**
     * Record audit event and anchor to blockchain
     */
    suspend fun recordAuditEvent(
        event: AuditEvent
    ): AuditRecord {

        // Create audit record
        val auditRecord = jsonData {
            "id" to event.id
            "timestamp" to event.timestamp.toString()
            "eventType" to event.type
            "actor" to event.actor
            "resource" to event.resource
            "details" to event.details
        }

        // Anchor to blockchain for immutability
        val anchored = trustWeave.blockchains.anchor(
            data = auditRecord,
            serializer = JsonObject.serializer(),
            chainId = "algorand:mainnet"
        )

        return AuditRecord(
            eventId = event.id,
            anchorRef = anchored.ref,
            timestamp = event.timestamp
        )
    }

    /**
     * Verify audit record integrity
     */
    suspend fun verifyAuditRecord(
        record: AuditRecord
    ): Boolean {
```

```
        // Read from blockchain
        val client = trustWeave.configuration.anchorClients[record.anchorRef.chainId]
            ?: return false

        val anchorResult = client.readPayload(record.anchorRef)

        // Verify integrity
        return anchorResult.payload.jsonObject["id"]?.jsonPrimitive?.content == record.eventId
    }
}
```

Deployment Strategy

Phase 1: MVP (Weeks 1-6)

1. **Setup TrustWeave**
    - Initialize TrustWeave with Algorand or Polygon
    - Create DIDs for EO providers
    - Setup blockchain anchoring
2. **Build SAR Flood Product**
    - Implement SAR flood detection
    - Create trigger evaluation logic
    - Build payout automation
3. **Broker Portal MVP**
    - Quote generation
    - Policy binding
    - Trigger dashboard

Phase 2: Production (Months 2-12)

1. **Add Heatwave Product**
2. **Add Solar Attenuation Product**
3. **Multi-provider EO data acceptance**
4. **Regulatory compliance features**
5. **Reinsurer dashboard**

Key Benefits of Using TrustWeave

1. **Standardization**: W3C-compliant format works with all EO providers
2. **Trust**: Cryptographic proof of data integrity
3. **Multi-Provider**: Accept data from ESA, Planet, NASA without custom integrations

4. **Regulatory Compliance**: Blockchain-anchored audit trails
5. **Automation**: Enable instant payouts with verifiable triggers
6. **Cost Reduction**: Eliminate custom API integrations

Next Steps

1. **Review Existing Scenarios**:
    - Parametric Insurance with EO Data
    - Earth Observation Scenario
2. **Explore TrustWeave APIs**:
    - Core API Reference
    - Blockchain Anchoring
3. **Start Building**:
    - Clone TrustWeave repository
    - Follow Quick Start Guide
    - Implement SAR flood product first

Related Documentation

- Parametric Insurance with EO Data - EO data insurance patterns
- Earth Observation Scenario - EO data integrity
- Blockchain Anchoring - Anchoring concepts
- API Reference - Complete API documentation